SAIR 代数推理竞赛项目工作历程与阶段性成果总结

截至日期：2026-03-29

阶段摘要（供快速阅读）

当前最佳结果来自稳态主分支 `P1_2_3_implicit_guardrail_v2`： 在 DeepSeek 的 `smoke` 固定切分上， `accuracy=0.5469` 、 `balanced accuracy=0.5371` 、 `parse success=1.0` 。这一结果的意义不在于已经得到最终高分，而在于项目已经找到一条兼顾“格式稳定可解析”和“真假例不过度失衡”的可复现主线；相比容易退化成全 `true` 或全 `false` 的分支， `P1_2_3` 更适合作为后续迭代基座。

`P1_2_3` 本质上是一条“隐式 guardrail”路线：不是额外堆一层很强的 false-filter，而是在较短 prompt 中用更轻的提醒，先保护塌缩类、常值类、直接蕴含类真例，再对缺乏支撑的目标保持谨慎。它是通过“公开数据接入 -> 固定切分 -> baseline -> 错误分析 -> 多轮 prompt 迭代 -> 候选分支对比”逐步筛出的；而官方 cheatsheet 样例直接 strict 化会退化为全 `false` ，因此当前主线仍以 `P1_2_3` 为准。

下一步转向“离线 family tagger 与结构规则资产沉淀”，指的是不再把每次发现的规律直接写回 prompt，而是先用程序把样本标成若干窄结构族，统计它们与真值、错误类型的关系，再把验证有效的规律沉淀为可复用规则资产，用于后续更隐式、更稳定的 prompt 利用。

1. 文档目的

本文档用于按照实际推进顺序，回顾本项目从启动到当前阶段已经完成的工作，并尽量用非专业人士也能理解的方式解释：

- 这个项目在做什么
- 为什么要这样组织工程
- 每一阶段做了哪些事情
- 这些事情分别解决了什么问题
- 目前取得了哪些阶段性结果
- 当前项目处在什么位置

需要特别说明的是，这个项目不是把竞赛当作一次性的“手工试题”，而是把它当作一个持续迭代的工程实验系统来建设。因此，除了 prompt 本身，我们还同步建设了数据链路、评测链路、错误分析、实验对比、版本管理和报告机制。

2. 项目背景：我们在解决什么问题

本竞赛的核心任务可以简单理解为：

给出两个代数等式 `Equation 1` 和 `Equation 2`，判断前者是否必然推出后者。

更口语化一点说，就是：

- 如果所有满足 `Equation 1` 的代数结构，也一定都满足 `Equation 2`，答案就是 `true`
- 只要能找到哪怕一个满足 `Equation 1` 但不满足 `Equation 2` 的代数结构，答案就是 `false`

这类题对大模型来说并不容易，因为它不是普通的文本匹配任务，而是同时涉及：

- 符号结构理解
- 变量与子式关系识别
- 抽象代数语义
- 多步推理
- 对“什么时候必须为真、什么时候只是不像真”的区分

因此，这个项目从一开始就不适合只靠“随便改几句 prompt”来推进，而需要一个稳定的工程框架，让每一轮尝试都可以被记录、复现和比较。

3. 项目启动阶段：研究输入整理与工程化实验计划制定

3.1 初始研究输入整理

项目启动时，首先不是写代码，而是整理已有研究材料和问题背景。核心输入包括：

- `docs/Doubao的总结与分析.md`
- `docs/大模型代数推理能力提升方法调研分析.md`
- `docs/大语言模型数学与代数推理能力提升的架构演进与前沿方法分析.md`
- `docs/大模型数学与代数推理能力提升方法综合报告.md`
- `webpages/` 下的网页资料

这一阶段的作用是把“零散知识”转化为“工程判断”。

当时形成的几个关键认识是：

1. 这个竞赛本质上是“等式蕴含判定”任务，而不是一般聊天问答。
2. 需要尽早建立标准数据集和固定评测集，否则每轮实验都无法公平比较。
3. 需要把 prompt 当作可版本化的实验对象来管理，而不是当作临时文本片段。

4. 需要建立错误分析体系，否则无法知道模型“为什么错”。

3.2 工程化实验计划制定

在研究输入的基础上，形成了总体实验计划文档：

- `docs/SAIR代数推理竞赛工程化实验计划.md`

这个计划文档的意义非常大。它不是简单的 TODO 列表，而是相当于项目的“施工图”。其中明确了：

- 仓库应该如何分层
- 数据该如何存放和切分
- baseline 应该先做哪些
- prompt 应该如何编号与对比
- 实验报告应该怎样产出
- 哪些内容属于竞赛私有信息，不能公开

如果把整个项目比作建房子，那么这一阶段做的事情就是先把图纸画出来，而不是先砌墙。

4. 工程仓库骨架阶段：把实验计划落成可运行仓库

4.1 建立统一目录结构

根据实验计划，完成了仓库骨架的初始化，建立了以下关键目录：

- `data/raw`
- `data/interim`
- `data/interim/splits`
- `data/external`
- `prompts/templates`
- `prompts/cheatsheets`
- `prompts/complete`
- `configs/models`
- `configs/eval`
- `configs/prompts`
- `reports/daily`
- `reports/experiments`
- `reports/release`
- `artifacts/candidates`
- `artifacts/final`

- `artifacts/backups`
- `src/sair_competition`
- `tests`

这样分层的原因是为了避免“所有东西堆在一起”。

不同目录承担的职责不同：

- `data/raw` 放官方原始数据，不改动
- `data/interim` 放清洗后的数据
- `prompts/templates` 放模板
- `prompts/cheatsheets` 放提示规则摘要
- `prompts/complete` 放可以直接测试或提交的完整 prompt
- `artifacts/candidates` 放实验运行结果
- `reports/experiments` 放实验结论和对比报告
- `src/sair_competition` 放实际代码

这样做的好处是，任何人进入仓库后都能很快判断“哪个文件是原始输入，哪个文件是中间产物，哪个文件是正式代码”。

4.2 Python 包与 CLI 工具落地

项目随后被做成了一个可执行的 Python 工具包：

- 包名： `sair-competition`
- CLI 入口： `sair`
- 主入口文件： `src/sair_competition/cli.py`

目前 CLI 已支持的主要命令包括：

- `validate-layout`
- `build-complete-prompt`
- `prepare-public-data`
- `make-splits`
- `run-baseline-eval`
- `run-complete-prompt-eval`
- `analyze-errors`
- `compare-candidates`

这一步非常重要，因为它把“人工操作”变成了“标准命令”。
标准命令的好处是：

- 每次执行流程都一致
- 更容易复现结果
- 更容易记录和自动化
- 以后可以继续扩展，而不需要推倒重来

5. 仓库私有化与运行环境治理：为什么要做 `.gitignore`

5.1 为什么竞赛项目必须重视私有化

竞赛项目和普通开源项目不同。普通开源项目强调尽量公开，竞赛项目则必须谨慎区分：

- 哪些是可以公开的“工程结构”
- 哪些是不能公开的“竞赛资产”

竞赛资产通常包括：

- 官方原始数据
- 自己的 prompt 文本
- 实验报告
- 中间分析结论
- 运行日志
- API 密钥

如果这些内容直接进入 Git 仓库，很容易误传到远端平台，造成不必要的风险。

5.2 `.gitignore` 的治理内容

因此，专门对 `.gitignore` 做了补强，目前已屏蔽：

- `.env` 与环境变量文件
- `webpages/` 研究资料
- `data/raw/*`
- `data/interim/*`
- `data/external/*`
- `prompts/cheatsheets/*`
- `prompts/complete/*`
- `reports/experiments/*`
- `reports/release/*`
- `artifacts/candidates/*`
- `artifacts/final/*`

- `artifacts/backups/*`
- 本地配置 JSON/JSONL
- pytest cache 与临时目录

但同时保留了：

- README
- 模板文件
- example config

这样一来，仓库公开时仍然能看出结构和方法，但不会暴露真正的竞赛内容。

5.3 运行缓存治理

在真实使用过程中，还遇到一个非常常见但很影响体验的问题：

- Python 和 pytest 运行后会自动在仓库根目录下生成 `__pycache__` 、pytest cache、临时目录等文件

这些文件虽然不影响程序功能，但会让仓库显得很乱，也更容易误提交。

后来采取了几项措施：

- 把 pytest cache 固定到 `artifacts/pytest_cache`
- 把 pytest 临时目录固定到 `artifacts/pytest_tmp`
- 在运行时设置 `PYTHONDONTWRITEBYTECODE=1`
- 在 `.gitignore` 中屏蔽常见缓存目录

这属于“工程卫生”工作。它不直接提升模型效果，但能让整个项目更稳定、更整洁，也更适合长期维护。

6. 公共数据接入阶段：官方数据标准化

6.1 为什么不能直接拿原始数据就跑

官方原始数据往往包含多个文件、多个格式和额外说明信息，直接拿来做实验会有几个问题：

- 格式不统一，不方便批量处理
- 难以统计总量和标签分布
- 不便于后续切分训练集/验证集/测试集
- 难以和其他辅助信息关联

因此需要先做“标准化”，也就是把分散在多个原始文件里的内容整理成统一结构。

6.2 已完成的数据标准化工作

这一阶段实现了公开数据接入与标准化，主要代码包括：

- `src/sair_competition/data/public_data.py`
- `src/sair_competition/data/io.py`
- `src/sair_competition/data/schemas.py`

标准化后的主数据文件为：

- `data/interim/public_all.jsonl`

同时生成了若干辅助注册表：

- `data/interim/benchmarks_registry.jsonl`
- `data/interim/prompt_templates_registry.jsonl`
- `data/interim/leaderboard_snapshot.jsonl`
- `data/interim/model_registry.jsonl`
- `data/interim/public_dataset_summary.json`

6.3 当前公开数据规模

标准化结果如下：

- 总题量： `1269`
- `normal` ： `1000`
- `hard1` ： `69`
- `hard2` ： `200`
- `true` ： `624`
- `false` ： `645`

这个分布有两个重要含义：

1. 标签整体比较平衡，没有严重偏向 `true` 或 `false` 。
   这意味着不能只靠“永远输出 false”来获得很高分。
2. 题目存在明显难度层级。
   这意味着后续分析不能只看总准确率，还要看不同难度来源上的表现。

7. 固定切分阶段：为什么要切分、为什么这样切、各集合分别做什么

这是很多非专业读者最容易疑惑的一部分，所以这里单独详细解释。

7.1 为什么要切分数据

如果把全部题目混在一起反复调 prompt，会出现一个严重问题：

- 你看到的结果越来越好，但你并不知道模型是真的变强了，还是只是“越来越适应你反复看到的那批题”

这在机器学习和实验设计里叫做“评测污染”或“过拟合风险”。

简单来说，如果同一批题既被拿来想规则，又被拿来打分，那么分数会越来越“不可信”。

因此，必须把数据切成不同用途的子集，让不同阶段做不同事情。

7.2 为什么要做“固定切分”

不仅要切分，而且要“固定切分”。

固定切分的意思是：

- 一旦确定了 `smoke/dev/holdout/audit` 的成员，就不要每次实验都随机重抽

原因是如果每轮都换一批题：

- 今天得分高，不知道是不是因为 prompt 真变好了
- 也可能只是今天抽到的题刚好更容易

固定切分的价值在于：

- 每次都是在同一组题上比较
- 差异可以更可信地归因到 prompt 或规则变化
- 报告里的对比更有意义

7.3 为什么切成 `smoke/dev/holdout/audit`

这里不是随便取了几个名字，而是对应不同工作阶段的不同用途。

`smoke`

作用：

- 最小规模、最快速的冒烟测试集

为什么需要它：

- 每次改 prompt 都跑全量会很慢、很贵
- 很多坏 prompt 一眼就能在小集合上看出来

它适合回答的问题是：

- 这版 prompt 是否能正常输出
- 是否出现了明显塌缩，例如几乎全 `false` 或几乎全 `true`
- 是否值得继续进入更大规模测试

当前 `smoke` 规模：

- 共 `64` 题
- `true=31`
- `false=33`
- 来源分布包含 `normal/hard1/hard2`

这意味着它是一个小而均衡的“体检集”。

`dev`

作用：

- 主要开发集

为什么需要它：

- `smoke` 太小，容易偶然波动
- 需要更大规模数据来观察 prompt 的稳定趋势

它适合回答的问题是：

- 这类规则在更大样本上是否还成立
- 某类错误是否真的在下降
- 某种改法是不是只在小样本上看起来有效

`holdout`

作用：

- 保留评估集

为什么需要它：

- 如果你经常盯着一批数据改 prompt，那批数据就不再是“客观评委”

`holdout` 的意义就是：

- 平时尽量少用
- 关键节点再用来检查“当前方法是否真的值得信”

它更像是项目中的“阶段考试”，不是每天练习题。

`audit`

作用：

- 额外人工抽查和分析集

为什么需要它：

- 有些时候我们不是只想知道“分数”，还想知道“错在哪”
- 需要一批适合做人工审阅、规则核查、边界案例复盘的样本

它更像“质检样本集”。

7.4 为什么是现在这个切分比例

当前切分如下：

- `smoke = 64`
- `dev = 824`
- `holdout = 254`
- `audit = 127`

这种安排背后的考虑是：

1. `smoke` 要足够小，才能频繁跑。
   如果 smoke 太大，每次迭代成本会过高。

2. `dev` 要足够大，才能承担主要实验工作。
   因为大部分迭代都要在这里看趋势。
3. `holdout` 要保留一定体量，才能在关键时刻做相对可信的验证。
   如果太小，就没有代表性。
4. `audit` 不需要特别大，但要足够做人工检查。
   它重在“可读”和“可分析”，不重在压分。

7.5 为什么切分后还要看标签分布和来源分布

切分不是把题目随机分出去就结束了，还要检查每个集合内部是否大致均衡。

如果一个集合里：

- `true` 太多
- 或 `false` 太多
- 或全是 `normal`
- 几乎没有 `hard`

那么这个集合就容易给出误导性结论。

当前 `smoke` 、 `dev` 、 `holdout` 、 `audit` 都保留了：

- `true/false` 的相对平衡
- `normal/hard1/hard2` 的来源覆盖

这保证了它们更像“缩小版总体”，而不是偏科样本。

8. Baseline 阶段：为什么先做 baseline，baseline 是怎么设计的

8.1 什么是 baseline

可以把 baseline 理解成“最基础参照物”。

在竞赛或模型实验中，如果没有 baseline，就会有两个问题：

1. 你不知道当前结果到底算好还是差。
2. 你不知道新方法到底是真的有效，还是只是比自己之前随手写的版本好一点。

所以 baseline 的作用不是追求最强，而是提供参照系。

8.2 为什么这个项目必须先做 baseline

这个项目的 prompt 设计很容易出现“感觉上更聪明，但实际没更好”的情况。
如果没有 baseline，就会经常出现：

- 看起来 reasoning 更长了
- prompt 更复杂了
- 但结果并没有比最简单的规则强

baseline 就像量尺，帮助我们判断：

- 一个复杂方案是否真的值这个复杂度
- 某个 prompt 是不是其实退回了很低水平

8.3 目前已经建立的 baseline

项目中已经实现了四个 baseline：

- `always_false`
- `always_true`
- `canonical_match`
- `structural_v1`

它们分别代表不同层级的“最低参照”。

`always_false`

意思很简单：

- 不管题目是什么，一律输出 `false`

这个 baseline 看似很笨，但它很有用。
因为如果数据里 `false` 稍微多一些，那么“永远输出 false”就可能拿到不算太差的总准确率。

所以它能告诉我们：

- 一个模型如果连 `always_false` 都赢不了，就说明它几乎没有真正学到东西

`always_true`

对应地：

- 不管题目是什么，一律输出 `true`

它的作用是对称参考，帮助我们看：

- 数据分布更偏哪边
- 模型是否出现了“全真塌缩”

`canonical_match`

这个 baseline 的意思是：

- 只有当两个等式在变量重命名后完全等价时，才输出 `true`

例如：

- `x = y * z`
- `a = b * c`

这类题在结构上是同一个模板，只是变量名字不同。

这个 baseline 的作用是提供一个“纯形式匹配”的下限：

- 它不会做真正推理
- 只会识别“其实就是同一句话”

`structural_v1`

这是目前 baseline 里最有实际价值的一个。
它不是大模型，也不懂真正的代数语义，但会使用一些轻量结构规则，例如：

- 比较两个等式的变量数量
- 比较运算符数量
- 比较左右侧结构形状
- 判断左右两边是否存在明显的“更强 / 更复杂”关系

它的意义在于：

- 它比“全 true / 全 false”更接近真实推理
- 它为后续 prompt 设计提供了结构启发
- 它能提醒我们：有些提升其实只是简单结构规则就能解释

8.4 baseline 是在哪个集合上来的

当前 baseline 的代表性结果主要是在 `holdout` 上统计的。

原因是：

- baseline 是项目的基础参照物
- 需要在一个相对独立、不过度参与日常调参的集合上给出结果

在 `holdout` 上， `structural_v1` 的表现为：

- Accuracy： `0.5787`
- True Accuracy： `0.4400`
- False Accuracy： `0.7132`

这个结果说明：

- 简单结构规则已经比“全真 / 全假”强不少
- 任务并不是完全只能靠大模型“玄学发挥”
- prompt 如果想证明自己有价值，至少要知道自己和这种结构启发式相比提升了什么

8.5 baseline 的真正作用

这套 baseline 后来承担了三种作用：

1. 作为“最低参照”。
   帮助判断一版 prompt 是否连最简单规则都不如。
2. 作为“结构启发来源”。
   帮助发现哪些结构特征值得写进 prompt 或错误分析。
3. 作为“实验 sanity check”。
   如果某个复杂 prompt 的表现和 `always_false` 很像，那往往说明它已经塌缩了。

9. 错误分析体系：为什么不能只看准确率

9.1 为什么只看总分会误导

如果只看一个数字，例如 Accuracy，很容易被误导。

举个简单例子：

- 一个 prompt 总准确率可能和另一个一样
- 但前者是“几乎全 false”
- 后者是“true 和 false 都还能判断一些”

这两者表面总分相同，但价值完全不同。

因此，项目里从比较早的时候就建立了错误分析体系，目的不是只看“错了多少”，而是看“错的类型是什么”。

9.2 当前错误分析的核心类别

当前错误分类体系包括：

- `FORMAT`
- `RULE_MISSING`
- `RULE_CONFLICT`
- `FALSE_FILTER_WEAK`
- `HARD_COMPOSITION`
- `PROMPT_AMBIGUOUS`
- `OVERCOMPRESSION`
- `MODEL_SPECIFIC`

其中在实际 prompt 迭代中最常用的是前五类。

9.3 外行可理解的错误类型解释

`FORMAT`

意思是：

- 模型输出格式不对，系统无法稳定解析成 `true` 或 `false`

这不是“推理错了”，而是“答题格式不合规”。

`RULE_MISSING`

意思是：

- 模型本来应该判 `true`
- 但 prompt 里缺少某条关键规则，导致它没能触发正确判断

可以把它理解成：

- 不是模型完全乱来
- 而是我们的提示规则没有覆盖到这类题

`RULE_CONFLICT`

意思是：

- prompt 里不同规则之间互相打架了
- 或者某条规则覆盖过头，误伤了别的情况

这类错误通常意味着：

- prompt 不是“规则太少”
- 而是“规则之间关系不对”

`FALSE_FILTER_WEAK`

意思是：

- prompt 太容易输出 `true`
- 没有足够谨慎地拦住一些其实不成立的题

直观理解就是：

- “放水太多”

`HARD_COMPOSITION`

意思是：

- 这道题不是靠一条单独规则就能搞定
- 需要多条规则组合，或者需要多步推理

这类错误通常出现在更难的题上，说明：

- prompt 不是完全没方向
- 而是模型没完成“把几步拼起来”的过程

9.4 错误分析体系的价值

这套体系后来成为 prompt 迭代的方向盘。

因为它能帮助我们回答：

- 这版 prompt 是规则不够，还是规则冲突
- 是过度保守，还是过度乐观
- 是结构理解问题，还是多步组合问题

如果没有这些分类，实验就只能停留在：

- “这版分数高一点”

- “那版分数低一点”

而无法知道下一步该改哪里。

10. Prompt 工程基础阶段：官方模板恢复与完整 prompt 机制

10.1 为什么要恢复官方模板

竞赛里 prompt 不是随便写的文本，而是实验对象本身。
如果连官方模板都没有恢复，那么后续就很难判断：

- 我们现在的实验和官方设置差了多少
- 哪些变化是基于官方模板进行的
- 哪些结果可以更好地对照官方公开信息

因此，根据 `data/raw/prompt_templates.jsonl` 恢复了：

- `prompts/templates/evaluation.jinja2`

这让仓库中有了一个可以追溯到官方记录的 prompt 模板。

10.2 为什么又建立了严格模板

官方模板更像“原始还原版”，但实际工程中还需要一个更适合自动化评测的版本，于是建立了：

- `prompts/templates/evaluation_strict_v1.jinja2`

它要求：

- 第一行只输出 `true` 或 `false`
- 不要额外标签
- 不要先写解释

原因很简单：

- 对人来说，多写解释可能没问题
- 对批量评测系统来说，格式越严格越稳定

10.3 为什么要分成 template / cheatsheet / complete prompt 三层

这是项目中非常关键的工程设计。

template

作用：

- 提供固定骨架，例如任务定义、输出格式、变量占位符

cheatsheet

作用：

- 放具体规则、提醒、推理约束

complete prompt

作用：

- 把 template 和 cheatsheet 合成后，得到可以直接送给模型的最终 prompt

这样设计的好处是：

1. 可以单独改规则，而不必每次重写整个 prompt。
2. 可以共享同一个模板，比较不同规则内容。
3. 更容易记录“这版 prompt 到底改了什么”。

这就像搭积木，而不是每次从头捏一个模型。

10.4 新增官方 cheatsheet 样例后，对“官方资产”的理解发生了什么变化

在最初阶段，我们拿到的、最容易直接利用的官方 prompt 相关材料，是 `data/raw/prompt_templates.jsonl` 中记录的模板信息。
因此当时项目的重点是先把官方 `evaluation` 模板恢复出来，再建立适合本地自动化评测的 strict 版本。

后来仓库中又加入了几份官方提供的 cheatsheet 样例：

- `data/raw/cheatsheet_balanced.md`
- `data/raw/cheatsheet_counterexample_first.md`
- `data/raw/cheatsheet_example_fast_filters.md`

这几份文件带来的一个重要认知更新是：

- 它们并不只是“几条简短规则的摘要”
- 它们通常同时包含任务 framing、占位符、推理步骤、输出格式要求

- 从工程上看，它们更接近“官方 prompt 原型”或“官方 prompt 风格样例”

这意味着我们对“官方资产”的理解，从原来的“只有官方 template 元数据”进一步扩展为：

- 一类资产是官方 template 元数据
- 另一类资产是官方 cheatsheet / prompt 风格样例

但这个变化并不意味着当前主线策略要被推翻。原因是这些官方样例往往：

- 更偏向完整解释型输出
- 容易要求模型生成 `REASONING` 、 `PROOF` 、 `COUNTEREXAMPLE` 等长文本
- 与当前 DeepSeek 本地 strict 评测链路中的“首行稳定输出 `true/false` ”目标并不完全一致

因此，到目前为止，这些官方样例的正确定位更适合是：

- 用来校准我们对官方 prompt 风格的理解
- 用来设计新的 prompt 家族实验分支
- 用来作为后续“官方风格 strict 改写版”的参考来源

而不是直接替换当前已经验证过稳定性的本地主线 prompt。

11. 本地评测链路阶段：DeepSeek 接入、解析器与运行问题修复

11.1 为什么要做本地 runner

如果每次测试 prompt 都靠手工复制粘贴到网页聊天框里，会有很多问题：

- 速度慢
- 不可复现
- 不方便批量统计
- 不方便保存原始输出
- 不方便做错误分析

因此，项目中实现了本地 runner：

- `src/sair_competition/eval/openai_compatible.py`
- `src/sair_competition/eval/local_runner.py`
- `src/sair_competition/config/env.py`

它允许我们通过 `.env` 中的 DeepSeek 配置，批量调用模型完成 smoke 评测。

11.2 为什么要特别做输出解析器

大模型即使知道答案，也不一定按你想要的格式输出。
它可能输出：

- `true`
- `TRUE`
- `VERDICT: TRUE`
- `The answer is false`

对于人来说，这些都能理解；但对自动脚本来说，必须明确怎么解析。

因此实现了输出解析器及测试：

- `src/sair_competition/eval/parser.py`
- `tests/test_eval_parser.py`
- `tests/test_eval_parser_more_formats.py`
- `tests/test_eval_parser_verdict.py`

这一步的作用是：

- 尽量降低格式波动带来的误判
- 让批量评测更稳定

11.3 运行环境问题修复

实际使用中遇到过：

- `No module named 'sair_competition'`

这类问题。后来明确了项目的标准运行方式：

- 安装 editable package
- 或在运行前设置 `PYTHONPATH=src`

并统一使用：

- `conda` 的 `DLEnv`

这一步看似技术细节，但非常关键，因为它解决的是“这个项目到底怎么稳定跑起来”的问题。

12. 第一轮 smoke 验证：打通实验闭环

12.1 为什么先做 smoke，而不是一上来跑全量

这和切分逻辑类似。
一个 prompt 刚写出来时，最常见的问题不是“差 1% 分数”，而是：

- 输出格式不对
- 整体塌缩成全 `false`
- 某条新规则把行为推歪了

这类问题没必要用大规模数据才能发现。
所以先用 `smoke` 做快速体检，成本最低，反馈最快。

12.2 最早的链路验证结果

使用示例 complete prompt 对 `smoke` 小样本做了首轮验证。
在 `limit=8` 的试跑中得到：

- Accuracy： `0.75`
- True Accuracy： `0.0`
- False Accuracy： `1.0`

这个结果并不表示模型很强，恰恰相反，它说明：

- 链路跑通了
- 但模型明显倾向于判 `false`

这正是 smoke 的价值：

- 不是要给出最终分数
- 而是尽快发现“当前行为长什么样”

12.3 第一批 prompt smoke 候选

随后陆续建立了早期候选：

- `P0_smoke_probe`
- `P1_1_0_strict_empty_probe`
- `P1_1_1_strict_first_draft_smoke`
- `P1_2_0_smoke_iter1`

- `P1_3_0_smoke_iter2`

其中最重要的阶段结论来自：

- `reports/experiments/smoke_prompt_compare/comparison.md`

12.4 第一阶段最重要的结果： `P1_2_0`

在早期实验中：

- `P1_1_1` 基本接近全 `false`
- `P1_3_0` 也退回全 `false`
- 只有 `P1_2_0_smoke_iter1` 真正打破了这种塌缩

`P1_2_0_smoke_iter1` 指标：

- Accuracy： `0.5156`
- True Accuracy： `0.6129`
- False Accuracy： `0.4242`
- Balanced Accuracy： `0.5186`

这一步的意义不在于它已经最好，而在于它第一次证明：

- 这个任务上，prompt 是可以被调出“非全 false”行为的
- 项目具备了继续优化的现实基础

13. 第二阶段：显式 false-filter 路线失败

13.1 当时为什么会想到做 false-filter

在 `P1_2_0` 后，分析发现一类明显问题：

- 有些题本来不应该判 `true`
- 但 prompt 在 shared-variable 情况下容易“过度相信结构相似性”

于是自然想到：

- 那就加一层更明确的“false-filter”来拦截这些假阳性

13.2 实验了哪些分支

这一阶段主要尝试了：

- `P1_2_1_family_d_light_false_filter`
- `P1_4_0_family_d_false_filter`
- `P1_4_1_family_d_false_filter_softened`
- `P1_4_2_family_d_false_filter_score`

对应报告：

- `reports/experiments/family_d_false_filter_compare/comparison.md`

13.3 为什么这条路被判定为失败

结果非常一致：

- 这些分支几乎都退化成全 `false`
- Accuracy 看上去没掉很多
- 但 True Accuracy 直接掉到 `0.0`

这说明：

- 它们并不是“更聪明地判断 false”
- 而是“把 true 也一起杀掉了”

这一步得出的模型行为结论非常重要：

- DeepSeek 对显式 false-filter 很敏感
- 一旦把“保守判 false”写得过重，就容易整体塌缩

这直接改变了后面的主线策略。

14. 第三阶段：implicit guardrail 路线成功建立稳态主分支

14.1 策略如何调整

既然显式 false-filter 会让模型塌缩，那么新的思路就是：

- 不把 false-filter 做成主决策层
- 只把它写成短、弱、隐式的 guardrail
- 保持 prompt 主体仍由正向规则和输出纪律构成

14.2 这一阶段的主要候选

- `P1_2_2_implicit_guardrail_v1`
- `P1_2_3_implicit_guardrail_v2`

对应报告：

- `reports/experiments/implicit_guardrail_compare/comparison.md`

14.3 为什么 `P1_2_3` 成为当前稳态主分支

`P1_2_3_implicit_guardrail_v2` 的结果为：

- Accuracy： `0.5469`
- True Accuracy： `0.2258`
- False Accuracy： `0.8485`
- Balanced Accuracy： `0.5371`

相较于 `P1_2_0`：

- Accuracy 增加 `+0.0312`
- Balanced Accuracy 增加 `+0.0186`

更重要的是，它没有像显式 false-filter 那样完全塌缩。
这说明：

- “短、弱、隐式 guardrail”是 DeepSeek 更能接受的风格

因此 `P1_2_3` 被保留为当前稳态主分支。

15. 第四阶段：围绕 `P1_2_3` 的高召回挖掘实验

15.1 为什么还要继续挖

虽然 `P1_2_3` 稳定，但它还有明显短板：

- 很多该判 `true` 的题仍被漏掉
- 特别是 `RULE_MISSING` 和 `HARD_COMPOSITION`

所以项目进入下一阶段：

- 不轻易动稳态主分支

- 但可以开更激进的高召回支线来“挖规则”

15.2 这一阶段的主要实验

- `P1_2_4_rule_missing_hard_composition`
- `P1_2_5_minimal_rule_missing_hard_composition`
- `P1_2_6_collapse_priority`
- `P1_2_7_light_anti_bias`
- `P1_2_8_narrow_singleton_families`

相关报告：

- `reports/experiments/P1_2_3_to_P1_2_7_iter_compare/comparison.md`
- `reports/experiments/P1_2_3_P1_2_5_P1_2_8_compare/comparison.md`

15.3 这些分支分别说明了什么

`P1_2_4`

对 rule-missing / hard-composition 的表达更直接，但再次把模型推向近似全 `false` 。
说明 DeepSeek 对 wording 非常敏感。

`P1_2_5`

这是这轮中最有价值的高召回支线：

- Accuracy： `0.5781`
- True Accuracy： `0.6452`
- False Accuracy： `0.5152`
- Balanced Accuracy： `0.5802`

它的意义是：

- 证明确实可以补回很多漏判的 `true`

但它的问题也很明显：

- 把不少不该通过的题也放过去了

所以它不适合作为主分支，更适合当“规则发现器”。

P1_2_6

把 collapse 优先级写得太强，模型又摆向近乎全 `true` 。

这说明：

- 不只是“过强 false-filter”危险
- “过强 true-trigger”同样危险

P1_2_7

哪怕只是更轻微的 anti-bias wording，DeepSeek 也可能退回近乎全 `false` 。

这进一步证明：

- `P1_2_3` 附近的 wording 稳定区间很窄

P1_2_8

把从 `P1_2_5` 中挖出的窄 singleton 结构族直接写回 prompt。
逻辑上它们是很干净的，但模型行为仍然被推回近似全 `false` 。

这说明：

- “规则逻辑上安全”不等于“写进 prompt 后模型还能稳定使用”

16. 规则挖掘阶段：从 `P1_2_5` 抽取真正有价值的结构族

16.1 为什么要把 `P1_2_5` 当成“挖掘器”

虽然 `P1_2_5` 不够稳，但它成功翻正了一批题。
这就意味着它并非没有价值，而是：

- 它包含了一些有用信息
- 只是这些信息目前还没有被“安全地”写成主 prompt

所以后续策略不是直接继承 `P1_2_5` ，而是把它当成：

- 高召回挖掘分支

也就是用它来回答：

- 它究竟补对了哪些题

- 这些题有没有共通结构
- 这些结构是否能在更大数据上验证为安全

16.2 已挖出的高置信结构族

在 `public_all` 上验证后，确认了两个很干净的 singleton 子族。

`singleton_left_same_lhs`

含义：

- `Equation 1` 是 `x = T`
- `x` 不出现在 `T` 里
- `Equation 2` 仍然以 `x` 为左边

统计结果：

- `161 true / 0 false`

这说明只要碰到这种结构，几乎可以确定它属于“安全的真例家族”。

`singleton_left_binary_lhs_with_x`

含义：

- `Equation 1` 是 `x = T`
- `x` 不出现在 `T` 里
- `Equation 2` 左边虽然不是单变量，但仍然包含 `x`

统计结果：

- `82 true / 0 false`

这类结构也非常干净。

16.3 为什么这些规则没有直接写回主 prompt

这是一个很关键、也很反直觉的结论。

按逻辑看，这些规则非常安全；但在 DeepSeek 上，一旦把它们写得更显式，模型整体行为却可能被推回近似全 `false` 。

所以目前项目采取的策略是：

- 把它们沉淀成“离线规则资产”
- 记录在本地规则注册表中
- 未来更可能通过 tagger、feature 或更隐式的利用方式使用
- 而不是直接改坏当前稳态主 prompt

这也是工程化方法的一个体现：

- 我们不只看“规则本身对不对”
- 还看“模型能不能稳定地按我们想要的方式使用它”

17. 当前仓库已经具备的能力

到目前为止，仓库已经形成了比较完整的竞赛工程能力。

17.1 数据能力

- 官方公开数据标准化
- 统一主数据文件生成
- 辅助 registry 生成
- 固定随机种子切分
- split manifest 记录

17.2 评测能力

- baseline 批量评测
- 本地 DeepSeek prompt 批量评测
- 输出解析
- 指标计算
- smoke / holdout 对照分析

17.3 Prompt 工程能力

- 官方模板恢复
- 严格模板
- cheatsheet / complete prompt 管理
- prompt 版本命名与比较

17.4 分析能力

- 错误类型分类

- 候选实验对比
- 分阶段实验报告
- 高召回分支的真例结构族挖掘

17.5 工程治理能力

- `.gitignore` 私有化治理
- 缓存与临时目录治理
- 统一运行环境规范
- 测试基础设施

18. 当前阶段的总体结论

如果按“从 0 到 1”的视角来看，本项目已经完成了第一轮完整建设。

这里的“完成”不是说已经拿到最终最优解，而是说：

1. 已经从研究与设想阶段进入可运行工程阶段。
2. 已经建立了可复现的数据、评测、对比和报告体系。
3. 已经摸清了 DeepSeek 在该任务上的几个关键行为边界。
4. 已经形成了一个当前可用的稳态主分支。
5. 已经形成了一个高召回规则挖掘支线。
6. 已经开始从“直接改 prompt”向“抽取结构规则资产”过渡。

18.1 最近一次补充实验：离线 family tagger 与官方原型 strict smoke 对照

在前面的工作基础上，项目又补做了一轮非常关键的实验。
这轮实验要回答两个问题：

1. 之前提出的“继续保留 `P1_2_3` 主线，并转向离线 family tagger / 数据打标”是否值得继续做。
2. 新加入仓库的官方 `cheatsheet_balanced`、`cheatsheet_counterexample_first`、`cheatsheet_example_fast_filters` 样例，是否应该直接改写后进入主线。

这轮实验的目的，不是追求立刻刷分，而是验证下一步路线是否正确。

18.2 这轮实验具体做了什么

这次补充实验分成两部分。

第一部分是“离线 family tagger”。

- 为每个样本自动打结构标签，例如：
  - `EQ1_SINGLETON_COLLAPSE_LHS`
  - `EQ1_DISJOINT_SIDES`
  - `EQ1_CONSTANT_OPERATION_CANDIDATE`
  - `SHARED_LHS_ALPHA`
  - `EQ2_INTRODUCES_NEW_VARS`
- 在 `smoke` 固定切分上生成带标签的数据版本：
  - `data/interim/splits/smoke_tagged.jsonl`
- 同时生成标签统计报告：
  - `reports/experiments/smoke_family_tags/summary.md`

第二部分是“官方原型 strict 适配对照实验”。

- 以官方三个样例为原型，分别做了三条 strict 化实验支线：
  - balanced
  - counterexample-first
  - fast-filters
- 这些支线不是直接替换主线，而是作为受控对照实验。
- 三条支线都在同一个 `smoke` 固定切分上、使用同一个 DeepSeek 配置运行。
- 最后与当前稳态主分支 `P1_2_3_implicit_guardrail_v2` 做对比，产出：
  - `reports/experiments/official_archetype_smoke_compare/comparison.md`

18.3 这轮实验得到了什么结果

先看离线 family tagger 的结果。

- `smoke` 集共有 `64` 题，其中 `48` 题被成功打上了结构标签。
- 其中最重要的三个标签是：
  - `EQ1_SINGLETON_COLLAPSE_LHS`： `14` 题，全部是真例
  - `EQ1_DISJOINT_SIDES`： `15` 题，全部是真例
  - `EQ1_CONSTANT_OPERATION_CANDIDATE`： `15` 题，全部是真例

这说明此前从 `P1_2_5` 等支线中抽出来的窄结构族，并不是偶然现象，而是确实能在公开样本中形成稳定分布的“高价值真例家族”。

再看官方原型 strict 支线的结果。

- 当前最佳主线 `P1_2_3_implicit_guardrail_v2`：
  - `accuracy = 0.5469`

    - `balanced accuracy = 0.5371`
    - `parse success = 1.0`
- 三条官方原型 strict 支线结果完全一致：
    - `accuracy = 0.5156`
    - `balanced accuracy = 0.5000`
    - `parse success = 1.0`
    - `true_accuracy = 0.0`
    - `false_accuracy = 1.0`

换句话说，三条官方原型 strict 支线虽然格式都能稳定解析，但都退化成了几乎“全 false”的行为模式。

从错误分析看，这三条支线的主要问题也高度一致：

- `RULE_MISSING = 25`
- `HARD_COMPOSITION = 6`

并且它们在下面这些标签家族上出现了集中漏判：

- `EQ1_SINGLETON_COLLAPSE_LHS`
- `EQ1_DISJOINT_SIDES`
- `EQ1_CONSTANT_OPERATION_CANDIDATE`

这说明这些官方样例的原始风格，哪怕做了 strict 化，在当前 DeepSeek 代理环境下仍然会把许多本该判 `true` 的“塌缩类 / 常值类”真例压成 `false` 。

18.4 这轮实验带来的判断

这次实验非常重要，因为它不是只告诉我们“哪个版本更高”，而是明确回答了“下一步该怎么走”。

可以确定的结论是：

1. 继续保留 `P1_2_3` 作为稳态主分支是正确的。
2. 继续保留 `P1_2_5` 作为高召回挖掘支线也是正确的，但它的价值主要在于抽取真例结构，不在于直接继承 wording。
3. “离线 family tagger / 数据打标”这条线值得继续推进，因为它已经开始把零散经验转化成可统计、可复用的结构规则资产。
4. 官方 cheatsheet 样例不是没用，但它们目前更适合作为“官方 prompt 原型参考”，而不是直接改写后进入主线。

因此，这轮实验实际上把项目从“继续堆 prompt wording”进一步推向了“用程序化标签和结构规则来辅助 prompt 迭代”的方向。

19. 当前项目状态与下一步方向

如果用一句话概括当前项目状态，可以表述为：

项目已经完成“工程底座建设 + 第一轮有效 prompt 路线探索 + 初步规则挖掘”，当前正处于从“直接改写 prompt”逐步过渡到“通过更隐式、更程序化的方式利用规则资产”的阶段。

当前明确的策略是：

- 保留 `P1_2_3_implicit_guardrail_v2` 作为稳态主分支
- 保留 `P1_2_5_minimal_rule_missing_hard_composition` 作为高召回挖掘支线
- 将已挖出的窄结构族优先作为离线规则资产使用
- 将官方 cheatsheet 样例视为“官方 prompt 原型参考”，单独规划 strict 适配实验支线，而不是直接并入主线

这意味着下一步工作的重点，不再只是“继续堆更多 wording”，而更可能转向：

- 离线 family tagger
- 更细粒度的样本标注
- 更隐式的规则注入方式
- 更稳定的 prompt 利用方式
- 官方 balanced / counterexample-first / fast-filters 风格的 strict 复刻与对照实验

从项目管理角度看，这标志着项目已经进入第二阶段：

- 第一阶段是“把系统搭起来，并找到能工作的主线”
- 第二阶段将是“如何在不破坏稳定性的前提下，继续放大正确规则的收益”